

```

    {
        if (propertyAValue != m_propertyAValue)
        {
            m_propertyAValue = propertyAValue;
            // Fire event. (See event example below.)
            PropertyChangedEvent(this, propertyAValue);
        }
    }
}

// Trivial get/set property that has a compiler-generated backing store.
property Platform::String^ PropertyB;

private:
    // Backing store for propertyA.
    int m_propertyAValue;
};

```

JavaScript

```

var nativeObject = new CppComponent.PropertyExample();
var propValue = nativeObject.propertyA;
document.getElementById('P8').innerHTML = propValue;

//Set the string property
nativeObject.propertyB = "What is the meaning of the universe?";
document.getElementById('P9').innerHTML += nativeObject.propertyB;

```

The .NET languages access properties on a native C++/CX object just as they would on a .NET object.

C#

```

private void GetAProperty()
{
    // Get the value of the integer property
    // Instantiate the C++ object
    var obj = new CppComponent.PropertyExample();

    // Get an integer property
    var propValue = obj.PropertyA;
    ResultText.Text += propValue.ToString();

    // Set a string property
    obj.PropertyB = " What is the meaning of the universe?";
    ResultText.Text += obj.PropertyB;
}

```

Delegates and events

A delegate is a Windows Runtime type that represents a function object. You can use delegates in connection with events, callbacks, and asynchronous method calls to specify an action to be performed later. Like a function object, the delegate provides type-safety by enabling the compiler to verify the return type and parameter types of the function. The declaration of a delegate resembles a function signature, the implementation resembles a class definition, and the invocation resembles a function invocation.

Adding an event listener

You can use the event keyword to declare a public member of a specified delegate type. Client code subscribes to the event by using the standard mechanisms that are provided in the particular language.

C++

```
public:
    event SomeHandler^ someEvent;
```

This example uses the same C++ code as for the previous properties section.

JavaScript

```
function Button_Click() {
    var nativeObj = new CppComponent.PropertyExample();
    // Define an event handler method
    var singlecasthandler = function (ev) {
        document.getElementById('P10').innerHTML = "The button was clicked
and the value is " + ev;
    };

    // Subscribe to the event
    nativeObj.onpropertychangedevent = singlecasthandler;

    // Set the value of the property and fire the event
    var propValue = 21;
    nativeObj.propertyA = 2 * propValue;
}
```

In the .NET languages, subscribing to an event in a C++ component is the same as subscribing to an event in a .NET class:

C#

```
//Subscribe to event and call method that causes it to be fired.
private void TestMethod()
{
    var objWithEvent = new CppComponent.PropertyExample();
    objWithEvent.PropertyChangedEvent += objWithEvent_PropertyChangedEvent;

    objWithEvent.PropertyA = 42;
}

//Event handler method
private void objWithEvent_PropertyChangedEvent(object __param0, int
__param1)
{
    ResultText.Text = "the event was fired and the result is " +
        __param1.ToString();
}
```

Adding multiple event listeners for one event

JavaScript has an `addEventListener` method that enables multiple handlers to subscribe to a single event.

C++

```
public delegate void SomeHandler(Platform::String^ str);

public ref class LangSample sealed
{
public:
    event SomeHandler^ someEvent;
    property Platform::String^ PropertyA;

    // Method that fires an event
    void FireEvent(Platform::String^ str)
    {
        someEvent(Platform::String::Concat(str, PropertyA->ToString()));
    }
    //...
};
```

JavaScript

```
// Add two event handlers
var multicast1 = function (ev) {
    document.getElementById('P11').innerHTML = "Handler 1: " + ev.target;
};
var multicast2 = function (ev) {
```

```

    document.getElementById('P12').innerHTML = "Handler 2: " + ev.target;
};

var nativeObject = new CppComponent.LangSample();
//Subscribe to the same event
nativeObject.addEventListener("someevent", multicast1);
nativeObject.addEventListener("someevent", multicast2);

nativeObject.propertyA = "42";

// This method should fire an event
nativeObject.fireEvent("The answer is ");

```

In C#, any number of event handlers can subscribe to the event by using the += operator as shown in the previous example.

Enums

A Windows Runtime enum in C++/CX is declared by using public class enum; it resembles a scoped enum in standard C++.

C++

```

public enum class Direction {North, South, East, West};

public ref class EnumExampleClass sealed
{
public:
    property Direction CurrentDirection
    {
        Direction get(){return m_direction; }
    }

private:
    Direction m_direction;
};

```

Enum values are passed between C++/CX and JavaScript as integers. You can optionally declare a JavaScript object that contains the same named values as the C++/CX enum and use it as follows.

JavaScript

```

var Direction = { 0: "North", 1: "South", 2: "East", 3: "West" };
//. . .

var nativeObject = new CppComponent.EnumExampleClass();
var curDirection = nativeObject.currentDirection;

```

```
document.getElementById('P13').innerHTML =  
Direction[curDirection];
```

Both C# and Visual Basic have language support for enums. These languages see a C++ public enum class just as they would see a .NET enum.

Asynchronous methods

To consume asynchronous methods that are exposed by other Windows Runtime objects, use the [task Class \(Concurrency Runtime\)](#). For more information, see [Task Parallelism \(Concurrency Runtime\)](#).

To implement asynchronous methods in C++/CX, use the [create_async](#) function that's defined in `ppltasks.h`. For more information, see [Creating Asynchronous Operations in C++/CX for UWP apps](#). For an example, see [Walkthrough of creating a C++/CX Windows Runtime component, and calling it from JavaScript or C#](#). The .NET languages consume C++/CX asynchronous methods just as they would any asynchronous method that's defined in .NET.

Exceptions

You can throw any exception type that's defined by the Windows Runtime. You cannot derive custom types from any Windows Runtime exception type. However, you can throw `COMException` and provide a custom `HRESULT` that can be accessed by the code that catches the exception. There's no way to specify a custom Message in a `COMException`.

Debugging tips

When you debug a JavaScript solution that has a component DLL, you can set the debugger to enable either stepping through script, or stepping through native code in the component, but not both at the same time. To change the setting, select the JavaScript project node in Solution Explorer and then choose Properties, Debugging, Debugger Type.

Be sure to select appropriate capabilities in the package designer. For example, if you are attempting to open an image file in the user's Pictures library by using the Windows Runtime APIs, be sure to select the Pictures Library check box in the Capabilities pane of the manifest designer.

If your JavaScript code doesn't seem to be recognizing the public properties or methods in the component, make sure that in JavaScript you are using camel casing. For example, the LogCalc C++/CX method must be referenced as logCalc in JavaScript.

If you remove a C++/CX Windows Runtime component project from a solution, you must also manually remove the project reference from the JavaScript project. Failure to do so prevents subsequent debug or build operations. If necessary, you can then add an assembly reference to the DLL.

Related topics

- [Walkthrough of creating a C++/CX Windows Runtime component, and calling it from JavaScript or C#](#)

Walkthrough of creating a C++/CX Windows Runtime component, and calling it from JavaScript or C#

Article • 10/20/2022

ⓘ Note

This topic exists to help you maintain your C++/CX application. But we recommend that you use **C++/WinRT** for new applications. C++/WinRT is an entirely standard modern C++17 language projection for Windows Runtime (WinRT) APIs, implemented as a header-file-based library, and designed to provide you with first-class access to the modern Windows API. To learn how to create a Windows Runtime component using C++/WinRT, see [Windows Runtime components with C++/WinRT](#).

This walkthrough shows how to create a basic Windows Runtime component DLL that's callable from JavaScript, C#, or Visual Basic. Before you begin this walkthrough, make sure that you understand concepts such as the Abstract Binary Interface (ABI), ref classes, and the Visual C++ Component Extensions that make working with ref classes easier. For more information, see [Windows Runtime components with C++/CX](#) and [Visual C++ Language Reference \(C++/CX\)](#).

Creating the C++ component DLL

In this example, we create the component project first, but you could create the JavaScript project first. The order doesn't matter.

Notice that the main class of the component contains examples of property and method definitions, and an event declaration. These are provided just to show you how it's done. They are not required, and in this example, we'll replace all of the generated code with our own code.

To create the C++ component project

1. On the Visual Studio menu bar, choose **File, New, Project**.
2. In the **New Project** dialog box, in the left pane, expand **Visual C++** and then select the node for Universal Windows apps.

3. In the center pane, select **Windows Runtime Component** and then name the project WinRT_CPP.
4. Choose the **OK** button.

To add an activatable class to the component

An activatable class is one that client code can create by using a **new** expression (**New** in Visual Basic, or **ref new** in C++). In your component, you declare it as **public ref class sealed**. In fact, the Class1.h and .cpp files already have a ref class. You can change the name, but in this example we'll use the default name—Class1. You can define additional ref classes or regular classes in your component if they are required. For more information about ref classes, see [Type System \(C++/CX\)](#).

Add these `#include` directives to Class1.h:

C++

```
#include <collection.h>
#include <ppl.h>
#include <amp.h>
#include <amp_math.h>
```

collection.h is the header file for C++ concrete classes such as the Platform::Collections::Vector class and the Platform::Collections::Map class, which implement language-neutral interfaces that are defined by the Windows Runtime. The amp headers are used to run computations on the GPU. They have no Windows Runtime equivalents, and that's fine because they are private. In general, for performance reasons you should use ISO C++ code and standard libraries internally within the component; it's just the Windows Runtime interface that must be expressed in Windows Runtime types.

To add a delegate at namespace scope

A delegate is a construct that defines the parameters and return type for methods. An event is an instance of a particular delegate type, and any event handler method that subscribes to the event must have the signature that's specified in the delegate. The following code defines a delegate type that takes an int and returns void. Next the code declares a public event of this type; this enables client code to provide methods that are invoked when the event is fired.

Add the following delegate declaration at namespace scope in Class1.h, just before the Class1 declaration.

C++

```
public delegate void PrimeFoundHandler(int result);
```

If the code isn't lining up correctly when you paste it into Visual Studio, just press Ctrl+K+D to fix the indentation for the entire file.

To add the public members

The class exposes three public methods and one public event. The first method is synchronous because it always executes very fast. Because the other two methods might take some time, they are asynchronous so that they don't block the UI thread. These methods return `IAsyncOperationWithProgress` and `IAsyncActionWithProgress`. The former defines an async method that returns a result, and the latter defines an async method that returns void. These interfaces also enable client code to receive updates on the progress of the operation.

C++

```
public:

    // Synchronous method.
    Windows::Foundation::Collections::IVector<double>^
    ComputeResult(double input);

    // Asynchronous methods

    Windows::Foundation::IAsyncOperationWithProgress<Windows::Foundation::Collections::IVector<int>^, double>^
    GetPrimesOrdered(int first, int last);
    Windows::Foundation::IAsyncActionWithProgress<double>^
    GetPrimesUnordered(int first, int last);

    // Event whose type is a delegate "class"
    event PrimeFoundHandler^ primeFoundEvent;
```

To add the private members

The class contains three private members: two helper methods for the numeric computations and a `CoreDispatcher` object that's used to marshal the event invocations

from worker threads back to the UI thread.

C++

```
private:
    bool is_prime(int n);
    Windows::UI::Core::CoreDispatcher^ m_dispatcher;
```

To add the header and namespace directives

1. In Class1.cpp, add these #include directives:

C++

```
#include <ppltasks.h>
#include <concurrent_vector.h>
```

2. Now add these using statements to pull in the required namespaces:

C++

```
using namespace concurrency;
using namespace Platform::Collections;
using namespace Windows::Foundation::Collections;
using namespace Windows::Foundation;
using namespace Windows::UI::Core;
```

To add the implementation for ComputeResult

In Class1.cpp, add the following method implementation. This method executes synchronously on the calling thread, but it is very fast because it uses C++ AMP to parallelize the computation on the GPU. For more information, see C++ AMP Overview. The results are appended to a Platform::Collections::Vector<T> concrete type, which is implicitly converted to a Windows::Foundation::Collections::IVector<T> when it is returned.

C++

```
//Public API
IVector<double>^ Class1::ComputeResult(double input)
{
    // Implement your function in ISO C++ or
    // call into your C++ lib or DLL here. This example uses AMP.
    float numbers[] = { 1.0, 10.0, 60.0, 100.0, 600.0, 10000.0 };
    array_view<float, 1> logs(6, numbers);
```

```

// See http://msdn.microsoft.com/library/hh305254.aspx
parallel_for_each(
    logs.extent,
    [=] (index<1> idx) restrict(amp)
    {
        logs[idx] = concurrency::fast_math::log10(logs[idx]);
    }
);

// Return a Windows Runtime-compatible type across the ABI
auto res = ref new Vector<double>();
int len = safe_cast<int>(logs.extent.size());
for(int i = 0; i < len; i++)
{
    res->Append(logs[i]);
}

// res is implicitly cast to IVector<double>
return res;
}

```

To add the implementation for GetPrimesOrdered and its helper method

In Class1.cpp, add the implementations for GetPrimesOrdered and the is_prime helper method. GetPrimesOrdered uses a concurrent_vector class and a parallel_for function loop to divide up the work and use the maximum resources of the computer on which the program is running to produce results. After the results are computed, stored, and sorted, they are added to a Platform::Collections::Vector<T> and returned as Windows::Foundation::Collections::IVector<T> to client code.

Notice the code for the progress reporter, which enables the client to hook up a progress bar or other UI to show the user how much longer the operation is going to take. Progress reporting has a cost. An event must be fired on the component side and handled on the UI thread, and the progress value must be stored on each iteration. One way to minimize the cost is by limiting the frequency at which a progress event is fired. If the cost is still prohibitive, or if you can't estimate the length of the operation, then consider using a progress ring, which shows that an operation is in progress but doesn't show time remaining until completion.

C++

```

// Determines whether the input value is prime.
bool Class1::is_prime(int n)
{

```

```

    if (n < 2)
        return false;
    for (int i = 2; i < n; ++i)
    {
        if ((n % i) == 0)
            return false;
    }
    return true;
}

// This method computes all primes, orders them, then returns the ordered
// results.
IAsyncOperationWithProgress<IVector<int>^, double>^
Class1::GetPrimesOrdered(int first, int last)
{
    return create_async([this, first, last]
        (progress_reporter<double> reporter) -> IVector<int>^ {
        // Ensure that the input values are in range.
        if (first < 0 || last < 0) {
            throw ref new InvalidArgumentException();
        }
        // Perform the computation in parallel.
        concurrent_vector<int> primes;
        long operation = 0;
        long range = last - first + 1;
        double lastPercent = 0.0;

        parallel_for(first, last + 1, [this, &primes, &operation,
            range, &lastPercent, reporter](int n) {

            // Increment and store the number of times the parallel
            // loop has been called on all threads combined. There
            // is a performance cost to maintaining a count, and
            // passing the delegate back to the UI thread, but it's
            // necessary if we want to display a determinate progress
            // bar that goes from 0 to 100%. We can avoid the cost by
            // setting the ProgressBar IsDeterminate property to false
            // or by using a ProgressRing.
            if(InterlockedIncrement(&operation) % 100 == 0)
            {
                reporter.report(100.0 * operation / range);
            }

            // If the value is prime, add it to the local vector.
            if (is_prime(n)) {
                primes.push_back(n);
            }
        });

        // Sort the results.
        std::sort(begin(primes), end(primes), std::less<int>());
        reporter.report(100.0);

        // Copy the results to a Vector object, which is
        // implicitly converted to the IVector return type. IVector

```

```

        // makes collections of data available to other
        // Windows Runtime components.
        return ref new Vector<int>(primes.begin(), primes.end());
    });
}

```

To add the implementation for GetPrimesUnordered

The last step to create the C++ component is to add the implementation for the GetPrimesUnordered in Class1.cpp. This method returns each result as it is found, without waiting until all results are found. Each result is returned in the event handler and displayed on the UI in real time. Again, notice that a progress reporter is used. This method also uses the is_prime helper method.

C++

```

// This method returns no value. Instead, it fires an event each time a
// prime is found, and passes the prime through the event.
// It also passes progress info.
IAsyncActionWithProgress<double>^ Class1::GetPrimesUnordered(int first, int
last)
{
    auto window = Windows::UI::Core::CoreWindow::GetForCurrentThread();
    m_dispatcher = window->Dispatcher;

    return create_async([this, first, last](progress_reporter<double>
reporter) {
        // Ensure that the input values are in range.
        if (first < 0 || last < 0) {
            throw ref new InvalidArgumentException();
        }

        // In this particular example, we don't actually use this to store
        // results since we pass results one at a time directly back to
        // UI as they are found. However, we have to provide this variable
        // as a parameter to parallel_for.
        concurrent_vector<int> primes;
        long operation = 0;
        long range = last - first + 1;
        double lastPercent = 0.0;

        // Perform the computation in parallel.
        parallel_for(first, last + 1,
            [this, &primes, &operation, range, &lastPercent, reporter](int
n)

```

```

{
    // Store the number of times the parallel loop has been called
    // on all threads combined. See comment in previous method.
    if(InterlockedIncrement(&operation) % 100 == 0)
    {
        reporter.report(100.0 * operation / range);
    }

    // If the value is prime, pass it immediately to the UI thread.
    if (is_prime(n))
    {
        // Since this code is probably running on a worker
        // thread, and we are passing the data back to the
        // UI thread, we have to use a CoreDispatcher object.
        m_dispatcher->RunAsync( CoreDispatcherPriority::Normal,
            ref new DispatchedHandler([this, n, operation, range]()
            {
                this->primeFoundEvent(n);

            }, Platform::CallbackContext::Any));
    }
});
reporter.report(100.0);
});
}

```

Creating a JavaScript client app (Visual Studio 2017)

If you want to create a C# client, then you can skip this section.

ⓘ Note

Universal Windows Platform (UWP) projects using JavaScript are not supported in Visual Studio 2019. See [JavaScript and TypeScript in Visual Studio 2019](#). To follow along with this section, we recommend that you use Visual Studio 2017. See [JavaScript in Visual Studio 2017](#).

To create a JavaScript project

1. In Solution Explorer (in Visual Studio 2017; see **Note** above), open the shortcut menu for the Solution node and choose **Add, New Project**.

2. Expand JavaScript (it might be nested under **Other Languages**) and choose **Blank App (Universal Windows)**.
3. Accept the default name—App1—by choosing the **OK** button.
4. Open the shortcut menu for the App1 project node and choose **Set as Startup Project**.
5. Add a project reference to WinRT_CPP:
6. Open the shortcut menu for the References node and choose **Add Reference**.
7. In the left pane of the References Manager dialog box, select **Projects** and then select **Solution**.
8. In the center pane, select WinRT_CPP and then choose the **OK** button

To add the HTML that invokes the JavaScript event handlers

Paste this HTML into the <body> node of the default.html page:

HTML

```
<div id="LogButtonDiv">
    <button id="logButton">Logarithms using AMP</button>
</div>
<div id="LogResultDiv">
    <p id="logResult"></p>
</div>
<div id="OrderedPrimeButtonDiv">
    <button id="orderedPrimeButton">Primes using parallel_for with
sort</button>
</div>
<div id="OrderedPrimeProgress">
    <progress id="OrderedPrimesProgressBar" value="0" max="100"></progress>
</div>
<div id="OrderedPrimeResultDiv">
    <p id="orderedPrimes">
        Primes found (ordered):
    </p>
</div>
<div id="UnorderedPrimeButtonDiv">
    <button id="ButtonUnordered">Primes returned as they are produced.
</button>
</div>
<div id="UnorderedPrimeDiv">
    <progress id="UnorderedPrimesProgressBar" value="0" max="100">
</progress>
```

```

</div>
<div id="UnorderedPrime">
  <p id="unorderedPrimes">
    Primes found (unordered):
  </p>
</div>
<div id="ClearDiv">
  <button id="Button_Clear">Clear</button>
</div>

```

To add styles

In default.css, remove the body style and then add these styles:

CSS

```

#LogButtonDiv {
border: orange solid 1px;
-ms-grid-row: 1; /* default is 1 */
-ms-grid-column: 1; /* default is 1 */
}
#LogResultDiv {
background: black;
border: red solid 1px;
-ms-grid-row: 1;
-ms-grid-column: 2;
}
#UnorderedPrimeButtonDiv, #OrderedPrimeButtonDiv {
border: orange solid 1px;
-ms-grid-row: 2;
-ms-grid-column:1;
}
#UnorderedPrimeProgress, #OrderedPrimeProgress {
border: red solid 1px;
-ms-grid-column-span: 2;
height: 40px;
}
#UnorderedPrimeResult, #OrderedPrimeResult {
border: red solid 1px;
font-size:smaller;
-ms-grid-row: 2;
-ms-grid-column: 3;
-ms-overflow-style:scrollbar;
}

```

To add the JavaScript event handlers that call into the component DLL

Add the following functions at the end of the default.js file. These functions are called when the buttons on the main page are chosen. Notice how JavaScript activates the C++ class, and then calls its methods and uses the return values to populate the HTML labels.

JavaScript

```
var nativeObject = new WinRT_CPP.Class1();

function LogButton_Click() {

    var val = nativeObject.computeResult(0);
    var result = "";

    for (i = 0; i < val.length; i++) {
        result += val[i] + "<br/>";
    }

    document.getElementById('logResult').innerHTML = result;
}

function ButtonOrdered_Click() {
    document.getElementById('orderedPrimes').innerHTML = "Primes found  
(ordered): ";

    nativeObject.getPrimesOrdered(2, 10000).then(
        function (v) {
            for (var i = 0; i < v.length; i++)
                document.getElementById('orderedPrimes').innerHTML += v[i] +
" ";
        },
        function (error) {
            document.getElementById('orderedPrimes').innerHTML += " " +
error.description;
        },
        function (p) {
            var progressBar =
document.getElementById("OrderedPrimesProgressBar");
            progressBar.value = p;
        });
}

function ButtonUnordered_Click() {
    document.getElementById('unorderedPrimes').innerHTML = "Primes found  
(unordered): ";
    nativeObject.onprimefoundevent = handler_unordered;

    nativeObject.getPrimesUnordered(2, 10000).then(
        function () { },
        function (error) {
            document.getElementById("unorderedPrimes").innerHTML += " " +
error.description;
        },
    ),
}
```

```

        function (p) {
            var progressBar =
document.getElementById("UnorderedPrimesProgressBar");
            progressBar.value = p;
        });
    }

var handler_unordered = function (n) {
    document.getElementById('unorderedPrimes').innerHTML +=
n.target.toString() + " ";
};

function ButtonClear_Click() {

    document.getElementById('logResult').innerHTML = "";
    document.getElementById("unorderedPrimes").innerHTML = "";
    document.getElementById('orderedPrimes').innerHTML = "";
    document.getElementById("UnorderedPrimesProgressBar").value = 0;
    document.getElementById("OrderedPrimesProgressBar").value = 0;
}

```

Add code to add the event listeners by replacing the existing call to WinJS.UI.processAll in app.onactivated in default.js with the following code that implements event registration in a then block. For a detailed explanation of this, see [Create a "Hello, World" app \(JS\)](#).

JavaScript

```

args.setPromise(WinJS.UI.processAll().then( function completed() {
    var logButton = document.getElementById("logButton");
    logButton.addEventListener("click", LogButton_Click, false);
    var orderedPrimeButton = document.getElementById("orderedPrimeButton");
    orderedPrimeButton.addEventListener("click", ButtonOrdered_Click,
false);
    var buttonUnordered = document.getElementById("ButtonUnordered");
    buttonUnordered.addEventListener("click", ButtonUnordered_Click, false);
    var buttonClear = document.getElementById("Button_Clear");
    buttonClear.addEventListener("click", ButtonClear_Click, false);
}));

```

Press F5 to run the app.

Creating a C# client app

To create a C# project

1. In Solution Explorer, open the shortcut menu for the Solution node and then choose **Add, New Project**.
2. Expand Visual C# (it might be nested under **Other Languages**), select **Windows** and then **Universal** in the left pane, and then select **Blank App** in the middle pane.
3. Name this app CS_Client and then choose the **OK** button.
4. Open the shortcut menu for the CS_Client project node and choose **Set as Startup Project**.
5. Add a project reference to WinRT_CPP:
 - Open the shortcut menu for the **References** node and choose **Add Reference**.
 - In the left pane of the **References Manager** dialog box, select **Projects** and then select **Solution**.
 - In the center pane, select WinRT_CPP and then choose the **OK** button.

To add the XAML that defines the user interface

Copy the following code into the Grid element in MainPage.xaml.

XAML

```
<ScrollView>
    <StackPanel Width="1400">

        <Button x:Name="Button1" Width="340" Height="50"
Margin="0,20,20,20" Content="Synchronous Logarithm Calculation"
FontSize="16" Click="Button1_Click_1"/>
        <TextBlock x:Name="Result1" Height="100" FontSize="14">
</TextBlock>
        <Button x:Name="PrimesOrderedButton" Content="Prime Numbers
Ordered" FontSize="16" Width="340" Height="50" Margin="0,20,20,20"
Click="PrimesOrderedButton_Click_1"></Button>
        <ProgressBar x:Name="PrimesOrderedProgress"
IsIndeterminate="false" Height="40"></ProgressBar>
        <TextBlock x:Name="PrimesOrderedResult" MinHeight="100"
FontSize="10" TextWrapping="Wrap"></TextBlock>
        <Button x:Name="PrimesUnOrderedButton" Width="340" Height="50"
Margin="0,20,20,20" Click="PrimesUnOrderedButton_Click_1" Content="Prime
Numbers Unordered" FontSize="16"></Button>
        <ProgressBar x:Name="PrimesUnOrderedProgress"
IsIndeterminate="false" Height="40" ></ProgressBar>
```

```

        <TextBlock x:Name="PrimesUnOrderedResult" MinHeight="100"
FontSize="10" TextWrapping="Wrap"></TextBlock>

        <Button x:Name="Clear_Button" Content="Clear"
HorizontalAlignment="Left" Margin="0,20,20,20" VerticalAlignment="Top"
Width="341" Click="Clear_Button_Click" FontSize="16"/>
    </StackPanel>
</ScrollViewer>

```

To add the event handlers for the buttons

In Solution Explorer, open MainPage.xaml.cs. (The file might be nested under MainPage.xaml.) Add a using directive for System.Text, and then add the event handler for the Logarithm calculation in the MainPage class.

C#

```

private void Button1_Click_1(object sender, RoutedEventArgs e)
{
    // Create the object
    var nativeObject = new WinRT_CPP.Class1();

    // Call the synchronous method. val is an IList that
    // contains the results.
    var val = nativeObject.ComputeResult(0);
    StringBuilder result = new StringBuilder();
    foreach (var v in val)
    {
        result.Append(v).Append(System.Environment.NewLine);
    }
    this.Result1.Text = result.ToString();
}

```

Add the event handler for the ordered result:

C#

```

async private void PrimesOrderedButton_Click_1(object sender,
RoutedEventArgs e)
{
    var nativeObject = new WinRT_CPP.Class1();

    StringBuilder sb = new StringBuilder();
    sb.Append("Primes found (ordered): ");

    PrimesOrderedResult.Text = sb.ToString();

    // Call the asynchronous method
    var asyncOp = nativeObject.GetPrimesOrdered(2, 100000);
}

```

```

// Before awaiting, provide a lambda or named method
// to handle the Progress event that is fired at regular
// intervals by the asyncOp object. This handler updates
// the progress bar in the UI.
asyncOp.Progress = (asyncInfo, progress) =>
{
    PrimesOrderedProgress.Value = progress;
};

// Wait for the operation to complete
var asyncResult = await asyncOp;

// Convert the results to strings
foreach (var result in asyncResult)
{
    sb.Append(result).Append(" ");
}

// Display the results
PrimesOrderedResult.Text = sb.ToString();
}

```

Add the event handler for the unordered result, and for the button that clears the results so that you can run the code again.

C#

```

private void PrimesUnOrderedButton_Click_1(object sender, RoutedEventArgs e)
{
    var nativeObject = new WinRT_CPP.Class1();

    StringBuilder sb = new StringBuilder();
    sb.Append("Primes found (unordered): ");
    PrimesUnOrderedResult.Text = sb.ToString();

    // primeFoundEvent is a user-defined event in nativeObject
    // It passes the results back to this thread as they are produced
    // and the event handler that we define here immediately displays them.
    nativeObject.primeFoundEvent += (n) =>
    {
        sb.Append(n.ToString()).Append(" ");
        PrimesUnOrderedResult.Text = sb.ToString();
    };

    // Call the async method.
    var asyncResult = nativeObject.GetPrimesUnordered(2, 100000);

    // Provide a handler for the Progress event that the asyncResult
    // object fires at regular intervals. This handler updates the progress
    bar.
    asyncResult.Progress += (asyncInfo, progress) =>
    {

```

```

        PrimesUnOrderedProgress.Value = progress;
    };
}

private void Clear_Button_Click(object sender, RoutedEventArgs e)
{
    PrimesOrderedProgress.Value = 0;
    PrimesUnOrderedProgress.Value = 0;
    PrimesUnOrderedResult.Text = "";
    PrimesOrderedResult.Text = "";
    Result1.Text = "";
}

```

Running the app

Select either the C# project or JavaScript project as the startup project by opening the shortcut menu for the project node in Solution Explorer and choosing **Set As Startup Project**. Then press F5 to run with debugging, or Ctrl+F5 to run without debugging.

Inspecting your component in Object Browser (optional)

In Object Browser, you can inspect all Windows Runtime types that are defined in .winmd files. This includes the types in the Platform namespace and the default namespace. However, because the types in the Platform::Collections namespace are defined in the header file collections.h, not in a winmd file, they don't appear in Object Browser.

To inspect a component

1. On the menu bar, choose **View, Object Browser** (Ctrl+Alt+J).
2. In the left pane of the Object Browser, expand the WinRT_CPP node to show the types and methods that are defined on your component.

Debugging tips

For a better debugging experience, download the debugging symbols from the public Microsoft symbol servers:

To download debugging symbols

1. On the menu bar, choose **Tools, Options**.
2. In the **Options** dialog box, expand **Debugging** and select **Symbols**.
3. Select **Microsoft Symbol Servers** and then choose the **OK** button.

It might take some time to download the symbols the first time. For faster performance the next time you press F5, specify a local directory in which to cache the symbols.

When you debug a JavaScript solution that has a component DLL, you can set the debugger to enable either stepping through script or stepping through native code in the component, but not both at the same time. To change the setting, open the shortcut menu for the JavaScript project node in Solution Explorer and choose **Properties, Debugging, Debugger Type**.

Be sure to select appropriate capabilities in the package designer. You can open the package designer by opening the Package.appxmanifest file. For example, if you are attempting to programmatically access files in the Pictures folder, be sure to select the **Pictures Library** check box in the **Capabilities** pane of the package designer.

If your JavaScript code doesn't recognize the public properties or methods in the component, make sure that in JavaScript you are using camel casing. For example, the `ComputeResult` C++ method must be referenced as `computeResult` in JavaScript.

If you remove a C++ Windows Runtime component project from a solution, you must also manually remove the project reference from the JavaScript project. Failure to do so prevents subsequent debug or build operations. If necessary, you can then add an assembly reference to the DLL.

Related topics

- [Windows Runtime components with C++/CX](#)

Windows Runtime components with C# and Visual Basic

Article • 10/20/2022

You can use managed code to create your own Windows Runtime types and package them in a Windows Runtime component. You can use your component in Universal Windows Platform (UWP) apps that are written in C++, JavaScript, Visual Basic, or C#. This topic outlines the rules for creating a component, and discusses some aspects of .NET support for the Windows Runtime. In general, that support is designed to be transparent to the .NET programmer. However, when you create a component to use with JavaScript or C++, you need to be aware of differences in the way those languages support the Windows Runtime.

If you are creating a component for use only in UWP apps that are written in Visual Basic or C#, and the component does not contain UWP controls, then consider using the **Class Library** template instead of the **Windows Runtime Component** project template in Microsoft Visual Studio. There are fewer restrictions on a simple class library.

ⓘ Note

For C# developers writing desktop apps in .NET 6 or later, use C#/WinRT to author a Windows Runtime component. See [Author Windows Runtime components with C#/WinRT](#).

Declaring types in Windows Runtime components

Internally, the Windows Runtime types in your component can use any .NET functionality that's allowed in a UWP app. For more info, see [.NET for UWP apps](#).

Externally, the members of your types can expose only Windows Runtime types for their parameters and return values. The following list describes the limitations on .NET types that are exposed from a Windows Runtime component.

- The fields, parameters, and return values of all the public types and members in your component must be Windows Runtime types. This restriction includes the Windows Runtime types that you author as well as types that are provided by the Windows Runtime itself. It also includes a number of .NET types. The inclusion of these types is part of the support that .NET provides to enable the natural use of the Windows Runtime in managed code—your code appears to use familiar .NET types instead of the underlying Windows Runtime types. For example, you can use .NET primitive types such as **Int32** and **Double**, certain fundamental types such as **DateTimeOffset** and **Uri**, and some commonly used generic interface types such as **IEnumerable<T>** (**IEnumerable(Of T)** in Visual Basic) and **IDictionary<TKey,TValue>**. Note that the type arguments of these generic types

must be Windows Runtime types. This is discussed in the sections [Passing Windows Runtime types to managed code](#) and [Passing managed types to the Windows Runtime](#), later in this topic.

- Public classes and interfaces can contain methods, properties, and events. You can declare delegates for your events, or use the **EventHandler<T>** delegate. A public class or interface can't:
 - Be generic.
 - Implement an interface that is not a Windows Runtime interface (however, you can create your own Windows Runtime interfaces and implement them).
 - Derive from types that are not in the Windows Runtime, such as **System.Exception** and **System.EventArgs**.
- All public types must have a root namespace that matches the assembly name, and the assembly name must not begin with "Windows".

Tip. By default, Visual Studio projects have namespace names that match the assembly name. In Visual Basic, the Namespace statement for this default namespace is not shown in your code.

- Public structures can't have any members other than public fields, and those fields must be value types or strings.
- Public classes must be **sealed** (**NotInheritable** in Visual Basic). If your programming model requires polymorphism, then you can create a public interface, and implement that interface on the classes that must be polymorphic.

Debugging your component

If both your UWP app and your component are built with managed code, then you can debug them both at the same time.

When you're testing your component as part of a UWP app using C++, you can debug managed and native code at the same time. The default is native code only.

To debug both native C++ code and managed code

1. Open the shortcut menu for your Visual C++ project, and choose **Properties**.
2. In the property pages, under **Configuration Properties**, choose **Debugging**.
3. Choose **Debugger Type**, and in the drop-down list box change **Native Only** to **Mixed (Managed and Native)**. Choose **OK**.
4. Set breakpoints in native and managed code.

When you're testing your component as part of a UWP app using JavaScript, by default the solution is in JavaScript debugging mode. In Visual Studio, you can't debug JavaScript and managed code at the same time.

To debug managed code instead of JavaScript

1. Open the shortcut menu for your JavaScript project, and choose **Properties**.
2. In the property pages, under **Configuration Properties**, choose **Debugging**.
3. Choose **Debugger Type**, and in the drop-down list box change **Script Only** to **Managed Only**. Choose **OK**.
4. Set breakpoints in managed code and debug as usual.

Passing Windows Runtime types to managed code

As mentioned previously in the section [Declaring types in Windows Runtime components](#), certain .NET types can appear in the signatures of members of public classes. This is part of the support that .NET provides to enable the natural use of the Windows Runtime in managed code. It includes primitive types and some classes and interfaces. When your component is used from JavaScript, or from C++ code, it's important to know how your .NET types appear to the caller. See [Walkthrough of creating a C# or Visual Basic Windows Runtime component, and calling it from JavaScript](#) for examples with JavaScript. This section discusses commonly used types.

In .NET, primitive types such as the **Int32** structure have many useful properties and methods, such as the **TryParse** method. By contrast, primitive types and structures in the Windows Runtime only have fields. When you pass these types to managed code, they appear to be .NET types, and you can use the properties and methods of .NET types as you normally would. The following list summarizes the substitutions that are made automatically in the IDE:

- For the Windows Runtime primitives **Int32**, **Int64**, **Single**, **Double**, **Boolean**, **String** (an immutable collection of Unicode characters), **Enum**, **UInt32**, **UInt64**, and **Guid**, use the type of the same name in the **System** namespace.
- For **UInt8**, use **System.Byte**.
- For **Char16**, use **System.Char**.
- For the **IInspectable** interface, use **System.Object**.

If C# or Visual Basic provides a language keyword for any of these types, then you can use the language keyword instead.

In addition to primitive types, some basic, commonly used Windows Runtime types appear in managed code as their .NET equivalents. For example, suppose your JavaScript code uses the **Windows.Foundation.Uri** class, and you want to pass it to a C# or Visual Basic method. The equivalent type in managed code is the .NET **System.Uri** class, and that's the type to use for the method parameter. You can tell when a Windows Runtime type appears as a .NET type,

because IntelliSense in Visual Studio hides the Windows Runtime type when you're writing managed code, and presents the equivalent .NET type. (Usually the two types have the same name. However, note that the **Windows.Foundation.DateTime** structure appears in managed code as **System.DateTimeOffset** and not as **System.DateTime**.)

For some commonly used collection types, the mapping is between the interfaces that are implemented by a Windows Runtime type and the interfaces that are implemented by the corresponding .NET type. As with the types mentioned above, you declare parameter types by using the .NET type. This hides some differences between the types and makes writing .NET code more natural.

The following table lists the most common of these generic interface types, along with other common class and interface mappings. For a complete list of Windows Runtime types that .NET maps, see [.NET mappings of Windows Runtime types](#).

Windows Runtime	.NET
IIterable<T>	IEnumerable<T>
IVector<T>	ICollection<T>
IVectorView<T>	ReadOnlyCollection<T>
IMap<K, V>	IDictionary<TKey, TValue>
IMapView<K, V>	ReadOnlyDictionary<TKey, TValue>
IKeyValuePair<K, V>	KeyValuePair<TKey, TValue>
IBindableIterable	IEnumerable
IBindableVector	ICollection
Windows.UI.Xaml.Data.INotifyPropertyChanged	System.ComponentModel.INotifyPropertyChanged
Windows.UI.Xaml.Data.PropertyChangedEventHandler	System.ComponentModel.PropertyChangedEventHandler
Windows.UI.Xaml.Data.PropertyChangedEventArgs	System.ComponentModel.PropertyChangedEventArgs

When a type implements more than one interface, you can use any of the interfaces it implements as a parameter type or return type of a member. For example, you can pass or return a **Dictionary<int, string>** (**Dictionary(Of Integer, String)** in Visual Basic) as **IDictionary<int, string>**, **ReadOnlyDictionary<int, string>**, or **IEnumerable<System.Collections.Generic.KeyValuePair<TKey, TValue>>**.

Important

JavaScript uses the interface that appears first in the list of interfaces that a managed type implements. For example, if you return **Dictionary<int, string>** to JavaScript code, it appears as **IDictionary<int, string>** no matter which interface you specify as the return

type. This means that if the first interface doesn't include a member that appears on later interfaces, that member isn't visible to JavaScript.

In the Windows Runtime, **IMap<K, V>** and **IMapView<K, V>** are iterated by using **KeyValuePair**. When you pass them to managed code, they appear as **IDictionary<TKey, TValue>** and **IReadOnlyDictionary<TKey, TValue>**, so naturally you use **System.Collections.Generic.KeyValuePair<TKey, TValue>** to enumerate them.

The way interfaces appear in managed code affects the way types that implement these interfaces appear. For example, the **PropertySet** class implements **IMap<K, V>**, which appears in managed code as **IDictionary<TKey, TValue>**. **PropertySet** appears as if it implemented **IDictionary<TKey, TValue>** instead of **IMap<K, V>**, so in managed code it appears to have an **Add** method, which behaves like the **Add** method on .NET dictionaries. It doesn't appear to have an **Insert** method. You can see this example in the topic [Walkthrough of creating a C# or Visual Basic Windows Runtime component, and calling it from JavaScript](#).

Passing managed types to the Windows Runtime

As discussed in the previous section, some Windows Runtime types can appear as .NET types in the signatures of your component's members, or in the signatures of Windows Runtime members when you use them in the IDE. When you pass .NET types to these members or use them as the return values of your component's members, they appear to the code on the other side as the corresponding Windows Runtime type. For examples of the effects this can have when your component is called from JavaScript, see the "Returning managed types from your component" section in [Walkthrough of creating a C# or Visual Basic Windows Runtime component, and calling it from JavaScript](#).

Overloaded methods

In the Windows Runtime, methods can be overloaded. However, if you declare multiple overloads with the same number of parameters, you must apply the **Windows.Foundation.Metadata.DefaultOverloadAttribute** attribute to only one of those overloads. That overload is the only one you can call from JavaScript. For example, in the following code the overload that takes an **int** (**Integer** in Visual Basic) is the default overload.

C#

```
public string OverloadExample(string s)
{
    return s;
}

[Windows.Foundation.Metadata.DefaultOverload()]
public int OverloadExample(int x)
{
```

```
    return x;  
}
```

[IMPORTANT] JavaScript allows you to pass any value to **OverloadExample**, and coerces the value to the type that is required by the parameter. You can call **OverloadExample** with "forty-two", "42", or 42.3, but all those values are passed to the default overload. The default overload in the previous example returns 0, 42, and 42, respectively.

You can't apply the **DefaultOverloadAttribute** attribute to constructors. All the constructors in a class must have different numbers of parameters.

Implementing IStringable

Starting with Windows 8.1, the Windows Runtime includes an **IStringable** interface whose single method, **IStringable.ToString**, provides basic formatting support comparable to that provided by **Object.ToString**. If you do choose to implement **IStringable** in a public managed type that is exported in a Windows Runtime component, the following restrictions apply:

- You can define the **IStringable** interface only in a "class implements" relationship, such as the following code in C#:

C#

```
public class NewClass : IStringable
```

Or the following Visual Basic code:

VB

```
Public Class NewClass : Implements IStringable
```

- You can't implement **IStringable** on an interface.
- You can't declare a parameter to be of type **IStringable**.
- **IStringable** can't be the return type of a method, property, or field.
- You can't hide your **IStringable** implementation from base classes by using a method definition such as the following:

C#

```
public class NewClass : IStringable  
{  
    public new string ToString()  
    {  
        return "New ToString in NewClass";  
    }  
}
```

```
}  
}
```

Instead, the **IStringable.ToString** implementation must always override the base class implementation. You can hide a **ToString** implementation only by invoking it on a strongly typed class instance.

ⓘ Note

Under a variety of conditions, calls from native code to a managed type that implements **IStringable** or hides its **ToString** implementation can produce unexpected behavior.

Asynchronous operations

To implement an asynchronous method in your component, add "Async" to the end of the method name and return one of the Windows Runtime interfaces that represent asynchronous actions or operations: **IAsyncAction**, **IAsyncActionWithProgress<TProgress>**, **IAsyncOperation<TResult>**, or **IAsyncOperationWithProgress<TResult, TProgress>**.

You can use .NET tasks (the **Task** class and generic **Task<TResult>** class) to implement your asynchronous method. You must return a task that represents an ongoing operation, such as a task that is returned from an asynchronous method written in C# or Visual Basic, or a task that is returned from the **Task.Run** method. If you use a constructor to create the task, you must call its **Task.Start** method before returning it.

A method that uses **await** (**Await** in Visual Basic) requires the **async** keyword (**Async** in Visual Basic). If you expose such a method from a Windows Runtime component, apply the **async** keyword to the delegate that you pass to the **Run** method.

For asynchronous actions and operations that do not support cancellation or progress reporting, you can use the **WindowsRuntimeSystemExtensions.AsAsyncAction** or **AsAsyncOperation<TResult>** extension method to wrap the task in the appropriate interface. For example, the following code implements an asynchronous method by using the **Task.Run<TResult>** method to start a task. The **AsAsyncOperation<TResult>** extension method returns the task as a Windows Runtime asynchronous operation.

C#

```
public static IAsyncOperation<IList<string>> DownloadAsStringsAsync(string id)  
{  
    return Task.Run<IList<string>>(async () =>  
    {  
        var data = await DownloadDataAsync(id);  
        return ExtractStrings(data);  
    }).AsAsyncOperation();  
}
```

The following JavaScript code shows how the method could be called by using a [WinJS.Promise](#) object. The function that is passed to the then method is executed when the asynchronous call completes. The stringList parameter contains the list of strings that is returned by the **DownloadAsStringAsync** method, and the function does whatever processing is required.

JavaScript

```
function asyncExample(id) {  
  
    var result = SampleComponent.Example.downloadAsStringAsync(id).then(  
        function (stringList) {  
            // Place code that uses the returned list of strings here.  
        });  
}
```

For asynchronous actions and operations that support cancellation or progress reporting, use the [AsyncInfo](#) class to generate a started task and to hook up the cancellation and progress reporting features of the task with the cancellation and progress reporting features of the appropriate Windows Runtime interface. For an example that supports both cancellation and progress reporting, see [Walkthrough of creating a C# or Visual Basic Windows Runtime component, and calling it from JavaScript](#).

Note that you can use the methods of the **AsyncInfo** class even if your asynchronous method doesn't support cancellation or progress reporting. If you use a Visual Basic lambda function or a C# anonymous method, don't supply parameters for the token and **IProgress<T>** interface. If you use a C# lambda function, supply a token parameter but ignore it. The previous example, which used the `AsAsyncOperation<TResult>` method, looks like this when you use the [AsyncInfo.Run<TResult>\(Func<CancellationToken, Task<TResult>>\)](#) method overload instead.

C#

```
public static IAsyncOperation<IList<string>> DownloadAsStringsAsync(string id)  
{  
    return AsyncInfo.Run<IList<string>>(async (token) =>  
    {  
        var data = await DownloadDataAsync(id);  
        return ExtractStrings(data);  
    });  
}
```

If you create an asynchronous method that optionally supports cancellation or progress reporting, consider adding overloads that don't have parameters for a cancellation token or the **IProgress<T>** interface.

Throwing exceptions

You can throw any exception type that is included in the .NET for Windows apps. You can't declare your own public exception types in a Windows Runtime component, but you can declare and throw non-public types.

If your component doesn't handle the exception, a corresponding exception is raised in the code that called your component. The way the exception appears to the caller depends on the way the calling language supports the Windows Runtime.

- In JavaScript, the exception appears as an object in which the exception message is replaced by a stack trace. When you debug your app in Visual Studio, you can see the original message text displayed in the debugger exception dialog box, identified as "WinRT Information". You can't access the original message text from JavaScript code.

Tip. Currently, the stack trace contains the managed exception type, but we don't recommend parsing the trace to identify the exception type. Instead, use an HRESULT value as described later in this section.

- In C++, the exception appears as a platform exception. If the managed exception's HRESULT property can be mapped to the HRESULT of a specific platform exception, the specific exception is used; otherwise, a [Platform::COMException](#) exception is thrown. The message text of the managed exception is not available to C++ code. If a specific platform exception was thrown, the default message text for that exception type appears; otherwise, no message text appears. See [Exceptions \(C++/CX\)](#).
- In C# or Visual Basic, the exception is a normal managed exception.

When you throw an exception from your component, you can make it easier for a JavaScript or C++ caller to handle the exception by throwing a non-public exception type whose HRESULT property value is specific to your component. The HRESULT is available to a JavaScript caller through the exception object's number property, and to a C++ caller through the [COMException::HRESULT](#) property.

ⓘ Note

Use a negative value for your HRESULT. A positive value is interpreted as success, and no exception is thrown in the JavaScript or C++ caller.

Declaring and raising events

When you declare a type to hold the data for your event, derive from Object instead of from EventArgs, because EventArgs is not a Windows Runtime type. Use

[EventHandler<EventArgs>](#) as the type of the event, and use your event argument type as the generic type argument. Raise the event just as you would in a .NET application.

When your Windows Runtime component is used from JavaScript or C++, the event follows the Windows Runtime event pattern that those languages expect. When you use the component from C# or Visual Basic, the event appears as an ordinary .NET event. An example is provided in [Walkthrough of creating a C# or Visual Basic Windows Runtime component, and calling it from JavaScript](#).

If you implement custom event accessors (declare an event with the **Custom** keyword, in Visual Basic), you must follow the Windows Runtime event pattern in your implementation. See [Custom events and event accessors in Windows Runtime components](#). Note that when you handle the event from C# or Visual Basic code, it still appears to be an ordinary .NET event.

Next steps

After you've created a Windows Runtime component for your own use, you may find that the functionality it encapsulates is useful to other developers. You have two options for packaging a component for distribution to other developers. See [Distributing a managed Windows Runtime component](#).

For more information about Visual Basic and C# language features, and .NET support for the Windows Runtime, see [Visual Basic](#) and [C#](#) documentation.

Troubleshooting

Symptom	Remedy
In a C++/WinRT app, when consuming a C# Windows Runtime component that uses XAML, the compiler produces an error of the form <i>"'MyNamespace_XamlTypeInfo': is not a member of 'winrt::MyNamespace'"</i> —where <i>MyNamespace</i> is the name of the Windows Runtime component's namespace.	In <code>pch.h</code> in the consuming C++/WinRT app, add <code>#include <winrt/MyNamespace.MyNamespace_XamlTypeInfo.h></code> —replacing <i>MyNamespace</i> as appropriate.

Related topics

- [.NET for UWP apps](#)
- [Walkthrough of creating a C# or Visual Basic Windows Runtime component, and calling it from JavaScript](#)

Custom events and event accessors in Windows Runtime components

Article • 10/20/2022

.NET support for Windows Runtime components makes it easy to declare events components by hiding the differences between the Universal Windows Platform (UWP) event pattern and the .NET event pattern. However, when you declare custom event accessors in a Windows Runtime component, you must follow the pattern used in the UWP.

Registering events

When you register to handle an event in the UWP, the add accessor returns a token. To unregister, you pass this token to the remove accessor. This means that the add and remove accessors for UWP events have different signatures from the accessors you're used to.

Fortunately, the Visual Basic and C# compilers simplify this process: When you declare an event with custom accessors in a Windows Runtime component, the compilers automatically use the UWP pattern. For example, you get a compiler error if your add accessor doesn't return a token. .NET provides two types to support the implementation:

- The [EventRegistrationToken](#) structure represents the token.
- The [EventRegistrationTokenTable<T>](#) class creates tokens and maintains a mapping between tokens and event handlers. The generic type argument is the event argument type. You create an instance of this class for each event, the first time an event handler is registered for that event.

The following code for the `NumberChanged` event shows the basic pattern for UWP events. In this example, the constructor for the event argument object, `NumberChangedEventArgs`, takes a single integer parameter that represents the changed numeric value.

Note This is the same pattern the compilers use for ordinary events that you declare in a Windows Runtime component.

```

private EventRegistrationTokenTable<EventHandler<NumberChangedEventArgs>>
    m_NumberChangedTokenTable = null;

public event EventHandler<NumberChangedEventArgs> NumberChanged
{
    add
    {
        return
        EventRegistrationTokenTable<EventHandler<NumberChangedEventArgs>>
            .GetOrCreateEventRegistrationTokenTable(ref
m_NumberChangedTokenTable)
            .AddEventHandler(value);
    }
    remove
    {
        EventRegistrationTokenTable<EventHandler<NumberChangedEventArgs>>
            .GetOrCreateEventRegistrationTokenTable(ref
m_NumberChangedTokenTable)
            .RemoveEventHandler(value);
    }
}

internal void OnNumberChanged(int newValue)
{
    EventHandler<NumberChangedEventArgs> temp =
        EventRegistrationTokenTable<EventHandler<NumberChangedEventArgs>>
            .GetOrCreateEventRegistrationTokenTable(ref
m_NumberChangedTokenTable)
            .InvocationList;
    if (temp != null)
    {
        temp(this, new NumberChangedEventArgs(newValue));
    }
}

```

VB

```

Private m_NumberChangedTokenTable As _
    EventRegistrationTokenTable(Of EventHandler(Of NumberChangedEventArgs))

Public Custom Event NumberChanged As EventHandler(Of NumberChangedEventArgs)

    AddHandler(ByVal handler As EventHandler(Of NumberChangedEventArgs))
        Return EventRegistrationTokenTable(Of EventHandler(Of
NumberChangedEventArgs)).
            GetOrCreateEventRegistrationTokenTable(m_NumberChangedTokenTable).
                AddEventHandler(handler)
    End AddHandler

    RemoveHandler(ByVal token As EventRegistrationToken)
        EventRegistrationTokenTable(Of EventHandler(Of

```

```

NumberChangedEventArgs)).

GetOrCreateEventRegistrationTokenTable(m_NumberChangedTokenTable).
    RemoveEventHandler(token)
End RemoveHandler

RaiseEvent(ByVal sender As Class1, ByVal args As NumberChangedEventArgs)
    Dim temp As EventHandler(Of NumberChangedEventArgs) = _
        EventRegistrationTokenTable(Of EventHandler(Of
NumberChangedEventArgs)).

GetOrCreateEventRegistrationTokenTable(m_NumberChangedTokenTable).
    InvocationList
    If temp IsNot Nothing Then
        temp(sender, args)
    End If
End RaiseEvent
End Event

```

The static (Shared in Visual Basic) `GetOrCreateEventRegistrationTokenTable` method creates the event's instance of the `EventRegistrationTokenTable<T>` object lazily. Pass the class-level field that will hold the token table instance to this method. If the field is empty, the method creates the table, stores a reference to the table in the field, and returns a reference to the table. If the field already contains a token table reference, the method just returns that reference.

Important To ensure thread safety, the field that holds the event's instance of `EventRegistrationTokenTable<T>` must be a class-level field. If it is a class-level field, the `GetOrCreateEventRegistrationTokenTable` method ensures that when multiple threads try to create the token table, all threads get the same instance of the table. For a given event, all calls to the `GetOrCreateEventRegistrationTokenTable` method must use the same class-level field.

Calling the `GetOrCreateEventRegistrationTokenTable` method in the remove accessor and in the `RaiseEvent` method (the `OnRaiseEvent` method in C#) ensures that no exceptions occur if these methods are called before any event handler delegates have been added.

The other members of the `EventRegistrationTokenTable<T>` class that are used in the UWP event pattern include the following:

- The `AddEventHandler` method generates a token for the event handler delegate, stores the delegate in the table, adds it to the invocation list, and returns the token.

- The [RemoveEventHandler\(EventRegistrationToken\)](#) method overload removes the delegate from the table and from the invocation list.

Note The `AddEventHandler` and `RemoveEventHandler(EventRegistrationToken)` methods lock the table to help ensure thread safety.

- The [InvocationList](#) property returns a delegate that includes all the event handlers that are currently registered to handle the event. Use this delegate to raise the event, or use the methods of the `Delegate` class to invoke the handlers individually.

Note We recommend that you follow the pattern shown in the example provided earlier in this article, and copy the delegate to a temporary variable before invoking it. This avoids a race condition in which one thread removes the last handler, reducing the delegate to null just before another thread tries to invoke the delegate. Delegates are immutable, so the copy is still valid.

Place your own code in the accessors as appropriate. If thread safety is an issue, you must provide your own locking for your code.

C# users: When you write custom event accessors in the UWP event pattern, the compiler doesn't provide the usual syntactic shortcuts. It generates errors if you use the name of the event in your code.

Visual Basic users: In .NET, an event is just a multicast delegate that represents all the registered event handlers. Raising the event just means invoking the delegate. Visual Basic syntax generally hides the interactions with the delegate, and the compiler copies the delegate before invoking it, as described in the note about thread safety. When you create a custom event in a Windows Runtime component, you have to deal with the delegate directly. This also means that you can, for example, use the [MulticastDelegate.GetInvocationList](#) method to get an array that contains a separate delegate for each event handler, if you want to invoke the handlers separately.

Related topics

- [Events \(Visual Basic\)](#)
- [Events \(C# Programming Guide\)](#)
- [.NET for UWP apps Overview](#)
- [.NET for UWP apps](#)
- [Walkthrough of creating a C# or Visual Basic Windows Runtime component, and calling it from JavaScript](#)

Passing arrays to a Windows Runtime Component

Article • 10/20/2022

In the Windows Universal Platform (UWP), parameters are either for input or for output, never both. This means that the contents of an array that is passed to a method, as well as the array itself, are either for input or for output. If the contents of the array are for input, the method reads from the array but doesn't write to it. If the contents of the array are for output, the method writes to the array but doesn't read from it. This presents a problem for array parameters, because arrays in .NET are reference types, and the contents of an array are mutable even when the array reference is passed by value (**ByVal** in Visual Basic). The [Windows Runtime Metadata Export Tool \(Winmdexp.exe\)](#) requires you to specify the intended usage of the array if it is not clear from context, by applying the `ReadOnlyArrayAttribute` attribute or the `WriteOnlyArrayAttribute` attribute to the parameter. Array usage is determined as follows:

- For the return value or for an out parameter (a **ByRef** parameter with the [OutAttribute](#) attribute in Visual Basic) the array is always for output only. Do not apply the `ReadOnlyArrayAttribute` attribute. The `WriteOnlyArrayAttribute` attribute is allowed on output parameters, but it's redundant.

Caution The Visual Basic compiler does not enforce output-only rules. You should never read from an output parameter; it may contain **Nothing**. Always assign a new array.

- Parameters that have the **ref** modifier (**ByRef** in Visual Basic) are not allowed. `Winmdexp.exe` generates an error.
- For a parameter that is passed by value, you must specify whether the array contents are for input or output by applying either the [ReadOnlyArrayAttribute](#) attribute or the [WriteOnlyArrayAttribute](#) attribute. Specifying both attributes is an error.

If a method must accept an array for input, modify the array contents, and return the array to the caller, use a read-only parameter for the input and a write-only parameter (or the return value) for the output. The following code shows one way to implement this pattern:

```
C#
```

```
public int[] ChangeArray([ReadOnlyArray()] int[] input)
{
    int[] output = input.Clone();
    // Manipulate the copy.
    // ...
    return output;
}
```

VB

```
Public Function ChangeArray(<ReadOnlyArray> input() As Integer) As Integer()
    Dim output() As Integer = input.Clone()
    ' Manipulate the copy.
    ' ...
    Return output
End Function
```

We recommend that you make a copy of the input array immediately, and manipulate the copy. This helps ensure that the method behaves the same whether or not your component is called by .NET code.

Using components from managed and unmanaged code

Parameters that have the `ReadOnlyArrayAttribute` attribute or the `WriteOnlyArrayAttribute` attribute behave differently depending on whether the caller is written in native code or managed code. If the caller is native code (JavaScript or Visual C++ component extensions), the array contents are treated as follows:

- `ReadOnlyArrayAttribute`: The array is copied when the call crosses the application binary interface (ABI) boundary. Elements are converted if necessary. Therefore, any accidental changes the method makes to an input-only array are not visible to the caller.
- `WriteOnlyArrayAttribute`: The called method can't make any assumptions about the contents of the original array. For example, the array the method receives might not be initialized, or might contain default values. The method is expected to set the values of all the elements in the array.

If the caller is managed code, the original array is available to the called method, as it would be in any method call in .NET. Array contents are mutable in .NET code, so any changes the method makes to the array are visible to the caller. This is important to remember because it affects unit tests written for a Windows Runtime Component. If

the tests are written in managed code, the contents of an array will appear to be mutable during testing.

Related topics

- [ReadOnlyArrayAttribute](#)
- [WriteOnlyArrayAttribute](#)
- [Windows Runtime components with C# and Visual Basic](#)

Diagnosing Windows Runtime Component error conditions

Article • 10/20/2022

This article provides additional information about restrictions on Windows Runtime components written with managed code. It expands on the information that is provided in error messages from [Winmdexp.exe \(Windows Runtime Metadata Export Tool\)](#), and complements the information on restrictions that is provided in [Windows Runtime components with C# and Visual Basic](#).

This article doesn't cover all errors. The errors discussed here are grouped by general category, and each category includes a table of associated error messages. Search for message text (omitting specific values for placeholders) or for message number. If you don't find the information you need here, please help us improve the documentation by using the feedback button at the end of this article. Include the error message.

Alternatively, you can file a bug at the Microsoft Connect website.

Error message for implementing async interface provides incorrect type

Managed Windows Runtime components cannot implement the Universal Windows Platform (UWP) interfaces that represent asynchronous actions or operations ([IAsyncAction](#), [IAsyncActionWithProgress<TProgress>](#), [IAsyncOperation<TResult>](#), or [IAsyncOperationWithProgress<TResult, TProgress>](#)). Instead, .NET provides the [AsyncInfo](#) class for generating async operations in Windows Runtime components. The error message that Winmdexp.exe displays when you try to implement an async interface incorrectly refers to this class by its former name, `AsyncInfoFactory`. .NET no longer includes the `AsyncInfoFactory` class.

Error number	Message Text
WME1084	Type '{0}' implements Windows Runtime async interface '{1}'. Windows Runtime types cannot implement async interfaces. Please use the <code>System.Runtime.InteropServices.WindowsRuntime.AsyncInfoFactory</code> class to generate async operations for export to Windows Runtime.

Note The error messages that refer to the Windows Runtime use an old terminology. This is now referred to as the Universal Windows Platform (UWP). For example, Windows Runtime types are now called UWP types.

Missing references to mscorlib.dll or System.Runtime.dll

This issue occurs only when you use Winmdexp.exe from the command line. We recommend that you use the /reference option to include references to both mscorlib.dll and System.Runtime.dll from the .NET Framework core reference assemblies, which are located in "%ProgramFiles(x86)%\Reference Assemblies\Microsoft\Framework\.NETCore\v4.5" ("%ProgramFiles%\..." on a 32-bit computer).

Error number	Message Text
WME1009	No reference was made to mscorlib.dll. A reference to this metadata file is required in order to export correctly.
WME1090	Could not determine the core reference assembly. Please make sure mscorlib.dll and System.Runtime.dll is referenced using the /reference switch.

Operator overloading is not allowed

In a Windows Runtime Component written in managed code, you cannot expose overloaded operators on public types.

Note In the error message, the operator is identified by its metadata name, such as op_Addition, op_Multiply, op_ExclusiveOr, op_Implicit (implicit conversion), and so on.

Error number	Message Text
WME1087	'{0}' is an operator overload. Managed types cannot expose operator overloads in the Windows Runtime.

Constructors on a class have the same number of parameters

In the UWP, a class can have only one constructor with a given number of parameters; for example, you can't have one constructor that has a single parameter of type **String** and another that has a single parameter of type **int** (**Integer** in Visual Basic). The only workaround is to use a different number of parameters for each constructor.

Error number	Message Text
WME1099	Type '{0}' has multiple constructors with '{1}' argument(s). Windows Runtime types cannot have multiple constructors with the same number of arguments.

Must specify a default for overloads that have the same number of parameters

In the UWP, overloaded methods can have the same number of parameters only if one overload is specified as the default overload. See "Overloaded Methods" in [Windows Runtime components with C# and Visual Basic](#).

Error number	Message Text
WME1059	Multiple {0}-parameter overloads of '{1}.{2}' are decorated with Windows.Foundation.Metadata.DefaultOverloadAttribute.
WME1085	The {0}-parameter overloads of {1}.{2} must have exactly one method specified as the default overload by decorating it with Windows.Foundation.Metadata.DefaultOverloadAttribute.

Namespace errors and invalid names for the output file

In the Universal Windows Platform, all the public types in a Windows metadata (.winmd) file must be in a namespace that shares the .winmd file name, or in sub-namespaces of the file name. For example, if your Visual Studio project is named A.B (that is, your Windows Runtime Component is A.B.winmd), it can contain public classes A.B.Class1 and A.B.C.Class2, but not A.Class3 (WME0006) or D.Class4 (WME1044).

Note These restrictions apply only to public types, not to private types used in your implementation.

In the case of A.Class3, you can either move Class3 to another namespace or change the name of the Windows Runtime Component to A.winmd. Although WME0006 is a warning, you should treat it as an error. In the previous example, code that calls A.B.winmd will be unable to locate A.Class3.

In the case of D.Class4, no file name can contain both D.Class4 and classes in the A.B namespace, so changing the name of the Windows Runtime Component is not an option. You can either move D.Class4 to another namespace, or put it in another Windows Runtime Component.

The file system can't distinguish between uppercase and lowercase, so namespaces that differ by case are not allowed (WME1067).

Your component must contain at least one **public sealed** type (**Public NotInheritable** in Visual Basic). If not, you will get WME1042 or WME1043, depending on whether your component contains private types.

A type in a Windows Runtime Component cannot have a name that is the same as a namespace (WME1068).

Caution If you call Winmdexp.exe directly and don't use the /out option to specify a name for your Windows Runtime Component, Winmdexp.exe tries to generate a name that includes all the namespaces in the component. Renaming namespaces can change the name of your component.

Error number	Message Text
WME0006	'{0}' is not a valid winmd file name for this assembly. All types within a Windows Metadata file must exist in a sub namespace of the namespace that is implied by the file name. Types that do not exist in such a sub namespace cannot be located at runtime. In this assembly, the smallest common namespace that could serve as a filename is '{1}'.
WME1042	Input module must contain at least one public type that is located inside a namespace.
WME1043	Input module must contain at least one public type that is located inside a namespace. The only types found inside namespaces are private.
WME1044	A public type has a namespace ('{1}') that shares no common prefix with other namespaces ('{0}'). All types within a Windows Metadata file must exist in a sub namespace of the namespace that is implied by the file name.
WME1067	Namespace names cannot differ only by case: '{0}', '{1}'.
WME1068	Type '{0}' cannot have the same name as namespace '{1}'.

Exporting types that aren't valid Universal Windows Platform types

The public interface of your component must expose only UWP types. However, .NET provides mappings for a number of commonly used types that are slightly different in .NET and the UWP. This enables the .NET developer to work with familiar types instead of learning new ones. You can use these mapped .NET types in the public interface of your component. See "Declaring types in Windows Runtime components" and "Passing Universal Windows Platform types to managed code" in [Windows Runtime components with C# and Visual Basic](#), and [.NET mappings of Windows Runtime types](#).

Many of these mappings are interfaces. For example, [IList<T>](#) maps to the UWP interface [IVector<T>](#). If you use `List<string>` (`List(Of String)` in Visual Basic) instead of `IList<string>` as a parameter type, Winmdexp.exe provides a list of alternatives that includes all the mapped interfaces implemented by `List<T>`. If you use nested generic types, such as `List<Dictionary<int, string>>` (`List(Of Dictionary(Of Integer, String))` in Visual Basic), Winmdexp.exe offers choices for each level of nesting. These lists can become quite long.

In general, the best choice is the interface that is closest to the type. For example, for `Dictionary<int, string>`, the best choice is most likely `IDictionary<int, string>`.

Important JavaScript uses the interface that appears first in the list of interfaces that a managed type implements. For example, if you return `Dictionary<int, string>` to JavaScript code, it appears as `IDictionary<int, string>` no matter which interface you specify as the return type. This means that if the first interface doesn't include a member that appears on later interfaces, that member isn't visible to JavaScript.

Caution Avoid using the non-generic [IList](#) and [IEnumerable](#) interfaces if your component will be used by JavaScript. These interfaces map to [IBindableVector](#) and [IBindableIterator](#), respectively. They support binding for XAML controls, and are invisible to JavaScript. JavaScript issues the run-time error "The function 'X' has an invalid signature and cannot be called."

Error number	Message Text
WME1033	Method '{0}' has parameter '{1}' of type '{2}'. '{2}' is not a valid Windows Runtime parameter type.

Error number	Message Text
WME1038	Method '{0}' has a parameter of type '{1}' in its signature. Although this type is not a valid Windows Runtime type, it implements interfaces that are valid Windows Runtime types. Consider changing the method signature to use one of the following types instead: '{2}'.
WME1039	Method '{0}' has a parameter of type '{1}' in its signature. Although this generic type is not a valid Windows Runtime type, the type or its generic parameters implement interfaces that are valid Windows Runtime types. {2} > Note For {2}, Winmdexp.exe appends a list of alternatives, such as "Consider changing the type 'System.Collections.Generic.List<T>' in the method signature to one of the following types instead: 'System.Collections.Generic.IList<T>', System.Collections.Generic.IReadOnlyList<T>, System.Collections.Generic.IEnumerable<T>'."
WME1040	Method '{0}' has a parameter of type '{1}' in its signature. Instead of using a managed Task type, use Windows.Foundation.IAsyncAction, Windows.Foundation.IAsyncOperation, or one of the other Windows Runtime async interfaces. The standard .NET await pattern also applies to these interfaces. Please see System.Runtime.InteropServices.WindowsRuntime.AsyncInfo for more information about converting managed task objects to Windows Runtime async interfaces.

Structures that contain fields of disallowed types

In the UWP, a structure can contain only fields, and only structures can contain fields. Those fields must be public. Valid field types include enumerations, structures, and primitive types.

Error number	Message Text
WME1060	Structure '{0}' has field '{1}' of type '{2}'. '{2}' is not a valid Windows Runtime field type. Each field in a Windows Runtime structure can only be UInt8, Int16, UInt16, Int32, UInt32, Int64, UInt64, Single, Double, Boolean, String, Enum, or itself a structure.

Restrictions on arrays in member signatures

In the UWP, arrays in member signatures must be one-dimensional with a lower bound of 0 (zero). Nested array types such as `myArray[][]` (`myArray()()` in Visual Basic) are not allowed.

Note This restriction does not apply to arrays you use internally in your implementation.

Error number	Message Text
WME1034	Method '{0}' has an array of type '{1}' with non-zero lower bound in its signature. Arrays in Windows Runtime method signatures must have a lower bound of zero.
WME1035	Method '{0}' has a multi-dimensional array of type '{1}' in its signature. Arrays in Windows Runtime method signatures must be one dimensional.
WME1036	Method '{0}' has a nested array of type '{1}' in its signature. Arrays in Windows Runtime method signatures cannot be nested.

Array parameters must specify whether array contents are readable or writable

In the UWP, parameters must be read-only or write-only. Parameters cannot be marked **ref** (**ByRef** without the [OutAttribute](#) attribute in Visual Basic). This applies to the contents of arrays, so array parameters must indicate whether the array contents are read-only or write-only. The direction is clear for **out** parameters (**ByRef** parameter with the [OutAttribute](#) attribute in Visual Basic), but array parameters that are passed by value (**ByVal** in Visual Basic) must be marked. See [Passing arrays to a Windows Runtime Component](#).

Error number	Message Text
--------------	--------------

Error number	Message Text
WME1101	Method '{0}' has parameter '{1}' which is an array, and which has both {2} and {3}. In the Windows Runtime, the contents array parameters must be either readable or writable. Please remove one of the attributes from '{1}'.
WME1102	Method '{0}' has an output parameter '{1}' which is an array, but which has {2}. In the Windows Runtime, the contents of output arrays are writable. Please remove the attribute from '{1}'.
WME1103	Method '{0}' has parameter '{1}' which is an array, and which has either a System.Runtime.InteropServices.InAttribute or a System.Runtime.InteropServices.OutAttribute. In the Windows Runtime, array parameters must have either {2} or {3}. Please remove these attributes or replace them with the appropriate Windows Runtime attribute if necessary.
WME1104	Method '{0}' has parameter '{1}' which is not an array, and which has either a {2} or a {3}. Windows Runtime does not support marking non-array parameters with {2} or {3}.
WME1105	Method '{0}' has parameter '{1}' with a System.Runtime.InteropServices.InAttribute or System.Runtime.InteropServices.OutAttribute. Windows Runtime does not support marking parameters with System.Runtime.InteropServices.InAttribute or System.Runtime.InteropServices.OutAttribute. Please consider removing System.Runtime.InteropServices.InAttribute and replace System.Runtime.InteropServices.OutAttribute with 'out' modifier instead. Method '{0}' has parameter '{1}' with a System.Runtime.InteropServices.InAttribute or System.Runtime.InteropServices.OutAttribute. Windows Runtime only supports marking ByRef parameters with System.Runtime.InteropServices.OutAttribute, and does not support other usages of those attributes.
WME1106	Method '{0}' has parameter '{1}' which is an array. In the Windows Runtime, the contents of array parameters must be either readable or writable. Please apply either {2} or {3} to '{1}'.

Member with a parameter named "value"

In the UWP, return values are considered to be output parameters, and the names of parameters must be unique. By default, Winmdexp.exe gives the return value the name "value". If your method has a parameter named "value", you will get error WME1092.

There are two ways to correct this:

- Give your parameter a name other than "value" (in property accessors, a name other than "returnValue").

- Use the `ReturnValueNameAttribute` attribute to change the name of the return value, as shown here:

C#

```
using System.Runtime.InteropServices;
using System.Runtime.InteropServices.WindowsRuntime;

[return: ReturnValueName("average")]
public int GetAverage(out int lowValue, out int highValue)
```

VB

```
Imports System.Runtime.InteropServices
Imports System.Runtime.InteropServices.WindowsRuntime

Public Function GetAverage(<Out> ByRef lowValue As Integer, _
    <Out> ByRef highValue As Integer) As <ReturnValueName("average")>
    String
```

Note If you change the name of the return value, and the new name collides with the name of another parameter, you will get error WME1091.

JavaScript code can access the output parameters of a method by name, including the return value. For an example, see the [ReturnValueNameAttribute](#) attribute.

Error number	Message Text
WME1091	The method '{0}' has the return value named '{1}' which is the same as a parameter name. Windows Runtime method parameters and return value must have unique names.
WME1092	The method '{0}' has a parameter named '{1}' which is the same as the default return value name. Consider using another name for the parameter or use the <code>System.Runtime.InteropServices.WindowsRuntime.ReturnValueNameAttribute</code> to explicitly specify the name of the return value.

Note The default name is "returnValue" for property accessors and "value" for all other methods.

Related topics

- [Windows Runtime components with C# and Visual Basic](#)
- [Winmdexp.exe \(Windows Runtime Metadata Export Tool\)](#)

Distributing a managed Windows Runtime Component

Article • 10/20/2022

You can distribute your Windows Runtime Component by file copy. However, if your component consists of many files, installation can be tedious for your users. Also, errors in placing files or failure to set references might cause problems for them. You can package a complex component as a Visual Studio extension SDK, to make it easy to install and use. Users only need to set one reference for the entire package. They can easily locate and install your component by using the **Extensions and Updates** dialog box, as described in [Finding and Using Visual Studio Extensions](#).

Planning a distributable Windows Runtime Component

Choose unique names for binary files, such as your .winmd files. We recommend the following format to ensure uniqueness:

syntax

```
company.product.purpose.extension  
For example: Microsoft.Cpp.Build.dll
```

Your binary files will be installed in app packages, possibly with binary files from other developers. See "Extension SDKs" in [Create a Software Development Kit](#).

To decide how to distribute your component, consider how complex it is. An extension SDK or similar package manager is recommended when:

- Your component consists of multiple files.
- You provide versions of your component for multiple platforms (x86 and Arm, for example).
- You provide both debug and release versions of your component.
- Your component has files and assemblies that are used only at design time.

An extension SDK is particularly useful if more than one of the above are true.

Note For complex components, the NuGet package management system offers an open source alternative to extension SDKs. Like extension SDKs, NuGet enables you to create packages that simplify the installation of complex components. For a

comparison of NuGet packages and Visual Studio extension SDKs, see [Adding References Using NuGet Versus an Extension SDK](#).

Distribution by file copy

If your component consists of a single .winmd file, or a .winmd file and a resource index (.pri) file, you can simply make the .winmd file available for users to copy. Users can put the file wherever they want to in a project, use the **Add Existing Item** dialog box to add the .winmd file to the project, and then use the Reference Manager dialog box to create a reference. If you include a .pri file or an .xml file, instruct users to place those files with the .winmd file.

Note Visual Studio always produces a .pri file when you build your Windows Runtime Component, even if your project doesn't include any resources. If you have a test app for your component, you can determine whether the .pri file is used by examining the contents of the app package in the bin\debug\AppX folder. If the .pri file from your component doesn't appear there, you don't need to distribute it. Alternatively, you can use the [MakePRI.exe](#) tool to dump the resource file from your Windows Runtime Component project. For example, in the Visual Studio Command Prompt window, type: `makepri dump /if MyComponent.pri /of MyComponent.pri.xml` You can read more about .pri files in [Resource Management System \(Windows\)](#).

Distribution by extension SDK

A complex component usually includes Windows resources, but see the note about detecting empty .pri files in the previous section.

To create an extension SDK

1. Make sure you have the Visual Studio SDK installed. You can download the Visual Studio SDK from the [Visual Studio Downloads](#) [↗](#) page.
2. Create a new project using the VSIX Project template. You can find the template under Visual C# or Visual Basic, in the Extensibility category. This template is installed as part of the Visual Studio SDK. ([Walkthrough: Create an SDK using C# or Visual Basic](#) or [Walkthrough: Create an SDK using C++](#), demonstrates the use of this template in a very simple scenario.)
3. Determine the folder structure for your SDK. The folder structure begins at the root level of your VSIX project, with the **References**, **Redist**, and **DesignTime** folders.

- **References** is the location for binary files that your users can program against. The extension SDK creates references to these files in your users' Visual Studio projects.
- **Redist** is the location for other files that must be distributed with your binary files, in apps that are created by using your component.
- **DesignTime** is the location for files that are used only when developers are creating apps that use your component.

In each of these folders, you can create configuration folders. The permitted names are debug, retail, and CommonConfiguration. The CommonConfiguration folder is for files that are the same whether they're used by retail or debug builds. If you're only distributing retail builds of your component, you can put everything in CommonConfiguration and omit the other two folders.

In each configuration folder, you can provide architecture folders for platform-specific files. If you use the same files for all platforms, you can supply a single folder named neutral. You can find details of the folder structure, including other architecture folder names, in [Create a Software Development Kit](#). (That article discusses both platform SDKs and extension SDKs. You may find it useful to collapse the section about platform SDKs, to avoid confusion.)

4. Create an SDK manifest file. The manifest specifies name and version information, the architectures your SDK supports, .NET versions, and other information about the way Visual Studio uses your SDK. You can find details and an example in [Create a Software Development Kit](#).
5. Build and distribute the extension SDK. For in-depth information, including localizing and signing the VSIX package, see [Shipping Visual Studio Extensions](#).

Related topics

- [Create a Software Development Kit](#)
- [NuGet package management system](#) 
- [Resource Management System \(Windows\)](#)
- [Finding and Using Visual Studio Extensions](#)
- [MakePRI.exe command options](#)

.NET mappings of Windows Runtime types in UWP apps

Article • 10/20/2022

This article lists the mappings that .NET makes between Windows Runtime (WinRT) types and .NET types in Universal Windows Platform (UWP) apps written with managed code. In these apps, Visual Studio IntelliSense shows the .NET type instead of the WinRT type. For example, if a WinRT method takes a parameter of type **IVector<string>**, then IntelliSense shows a parameter of type **IList<string>**. Similarly, in a Windows Runtime component written with managed code, you use the .NET type in member signatures. When the [Windows Runtime Metadata Export Tool \(Winmdexp.exe\)](#) generates your Windows Runtime component, the .NET type is translated into the corresponding WinRT type.

Most of the types that have the same namespace name and type name in both WinRT and .NET are structures (or types associated with structures, such as enumerations). In WinRT, structures have no members other than fields, and require helper types, which .NET hides. The .NET versions of these structures have properties and methods that provide the functionality of the hidden helper types.

ⓘ Note

For a list of mappings between WinRT and .NET types in the context of desktop apps that target .NET and use C#/WinRT, see [.NET mappings of WinRT types in C#/WinRT](#).

WinRT types that map to .NET types with the same name and namespace

In .NET assembly System.ObjectModel.dll

Namespace	Type
Windows.UI.Xaml.Input	ICommand

In .NET assembly System.Runtime.WindowsRuntime.dll

Namespace	Type
Windows.Foundation	Point
Windows.Foundation	Rect
Windows.Foundation	Size
Windows.UI	Color

In .NET assembly System.Runtime.WindowsRuntime.UI.Xaml.dll

Namespace	Type
Windows.UI.Xaml	CornerRadius
Windows.UI.Xaml	Duration
Windows.UI.Xaml	DurationType
Windows.UI.Xaml	GridLength
Windows.UI.Xaml	GridUnitType
Windows.UI.Xaml	Thickness
Windows.UI.Xaml.Controls.Primitives	GeneratorPosition
Windows.UI.Xaml.Media	Matrix
Windows.UI.Xaml.Media.Animation	KeyTime
Windows.UI.Xaml.Media.Animation	RepeatBehavior
Windows.UI.Xaml.Media.Animation	RepeatBehaviorType
Windows.UI.Xaml.Media.Media3D	Matrix3D

WinRT types that map to .NET types with a different name and/or namespace

In .NET assembly System.ObjectModel.dll

WinRT type/namespace	.NET type/namespace
----------------------	---------------------

WinRT type/namespace	.NET type/namespace
INotifyCollectionChanged (Windows.UI.Xaml.Interop)	INotifyCollectionChanged (System.Collections.Specialized)
NotifyCollectionChangedEventHandler (Windows.UI.Xaml.Interop)	NotifyCollectionChangedEventHandler (System.Collections.Specialized)
NotifyCollectionChangedEventArgs (Windows.UI.Xaml.Interop)	NotifyCollectionChangedEventArgs (System.Collections.Specialized)
NotifyCollectionChangedAction (Windows.UI.Xaml.Interop)	NotifyCollectionChangedAction (System.Collections.Specialized)
INotifyPropertyChanged (Windows.UI.Xaml.Data)	INotifyPropertyChanged (System.ComponentModel)
PropertyChangedEventHandler (Windows.UI.Xaml.Data)	PropertyChangedEventHandler (System.ComponentModel)
PropertyChangedEventArgs (Windows.UI.Xaml.Data)	PropertyChangedEventArgs (System.ComponentModel)

In .NET assembly System.Runtime.dll

WinRT type/namespace	.NET type/namespace
AttributeUsageAttribute (Windows.Foundation.Metadata)	AttributeUsageAttribute (System)
AttributeTargets (Windows.Foundation.Metadata)	AttributeTargets (System)
DateTime (Windows.Foundation)	DateTimeOffset (System)
EventHandler<T> (Windows.Foundation)	EventHandler<T> (System)
HResult (Windows.Foundation)	Exception (System)
IReference<T> (Windows.Foundation)	Nullable<T> (System)
TimeSpan (Windows.Foundation)	TimeSpan (System)
Uri (Windows.Foundation)	Uri (System)
IClosable (Windows.Foundation)	IDisposable (System)
Iterable<T> (Windows.Foundation.Collections)	IEnumerable<T> (System.Collections.Generic)

WinRT type/namespace	.NET type/namespace
IVector<T> (Windows.Foundation.Collections)	ICollection<T> (System.Collections.Generic)
IVectorView<T> (Windows.Foundation.Collections)	ICollection<T> (System.Collections.Generic)
IMap<K,V> (Windows.Foundation.Collections)	IDictionary<TKey,TValue> (System.Collections.Generic)
IMapView<K,V> (Windows.Foundation.Collections)	ICollection<TKey,TValue> (System.Collections.Generic)
IKeyValuePair<K,V> (Windows.Foundation.Collections)	KeyValuePair<TKey,TValue> (System.Collections.Generic)
IBindableIterable (Windows.UI.Xaml.Interop)	IEnumerable (System.Collections)
IBindableVector (Windows.UI.Xaml.Interop)	ICollection (System.Collections)
TypeName (Windows.UI.Xaml.Interop)	Type (System)

In .NET assembly System.Runtime.InteropServices.WindowsRuntime.dll

WinRT type/namespace	.NET type/namespace
EventRegistrationToken (Windows.Foundation)	EventRegistrationToken (System.Runtime.InteropServices.WindowsRuntime)

Related topics

- [Windows Runtime components with C# and Visual Basic](#)

Walkthrough of creating a C# or Visual Basic Windows Runtime component, and calling it from JavaScript

Article • 10/20/2022

This walkthrough shows how you can use .NET with Visual Basic or C# to create your own Windows Runtime types, packaged in a Windows Runtime component, and how then to call that component from a JavaScript Universal Windows Platform (UWP) app.

Visual Studio makes it easy to author and deploy your own custom Windows Runtime types inside a Windows Runtime component (WRC) project written with C# or Visual Basic, and then to reference that WRC from a JavaScript application project, and to consume those custom types from that application.

Internally, your Windows Runtime types can use any .NET functionality that's allowed in a UWP application.

ⓘ Note

For more info, see [Windows Runtime components with C# and Visual Basic and .NET for UWP apps overview](#).

Externally, the members of your type can expose only Windows Runtime types for their parameters and return values. When you build your solution, Visual Studio builds your .NET WRC project, and then executes a build step that creates a Windows metadata (.winmd) file. This is your Windows Runtime component, which Visual Studio includes in your app.

ⓘ Note

.NET automatically maps some commonly used .NET types, such as primitive data types and collection types, to their Windows Runtime equivalents. These .NET types can be used in the public interface of a Windows Runtime component, and will appear to users of the component as the corresponding Windows Runtime types. See [Windows Runtime components with C# and Visual Basic](#).

Prerequisites:

- Windows 10
- [Microsoft Visual Studio](#) ↗

ⓘ Note

Universal Windows Platform (UWP) projects using JavaScript are not supported in Visual Studio 2019. See [JavaScript and TypeScript in Visual Studio 2019](#). To follow along with this topic, we recommend that you use Visual Studio 2017. See [JavaScript in Visual Studio 2017](#).

Creating a simple Windows Runtime class

This section creates a JavaScript UWP application, and adds to the solution a Visual Basic or C# Windows Runtime component project. It shows how to define a Windows Runtime type, create an instance of the type from JavaScript, and call static and instance members. The visual display of the example app is deliberately low-key in order to keep the focus on the component.

1. In Visual Studio, create a new JavaScript project: On the menu bar, choose **File, New, Project**. In the **Installed Templates** section of the **New Project** dialog box, choose **JavaScript**, and then choose **Windows**, and then **Universal**. (If Windows is not available, make sure you're using Windows 8 or later.) Choose the **Blank Application** template and enter **SampleApp** for the project name.
2. Create the component project: In Solution Explorer, open the shortcut menu for the **SampleApp** solution and choose **Add**, and then choose **New Project** to add a new C# or Visual Basic project to the solution. In the **Installed Templates** section of the **Add New Project** dialog box, choose **Visual Basic** or **Visual C#**, and then choose **Windows**, and then **Universal**. Choose the **Windows Runtime Component** template and enter **SampleComponent** for the project name.
3. Change the name of the class to **Example**. Notice that by default, the class is marked **public sealed** (**Public NotInheritable** in Visual Basic). All the Windows Runtime classes you expose from your component must be sealed.
4. Add two simple members to the class, a **static** method (**Shared** method in Visual Basic) and an instance property:

C#

```
namespace SampleComponent
{
```

```

public sealed class Example
{
    public static string GetAnswer()
    {
        return "The answer is 42.";
    }

    public int SampleProperty { get; set; }
}

```

VB

```

Public NotInheritable Class Example
    Public Shared Function GetAnswer() As String
        Return "The answer is 42."
    End Function

    Public Property SampleProperty As Integer
End Class

```

5. Optional: To enable IntelliSense for the newly added members, in Solution Explorer, open the shortcut menu for the SampleComponent project, and then choose **Build**.
6. In Solution Explorer, in the JavaScript project, open the shortcut menu for **References**, and then choose **Add Reference** to open the **Reference Manager**. Choose **Projects**, and then choose **Solution**. Select the check box for the SampleComponent project and choose **OK** to add a reference.

Call the component from JavaScript

To use the Windows Runtime type from JavaScript, add the following code in the anonymous function in the default.js file (in the js folder of the project) that is provided by the Visual Studio template. It should go after the app.oncheckpoint event handler and before the call to app.start.

JavaScript

```

var ex;

function basics1() {
    document.getElementById('output').innerHTML =
        SampleComponent.Example.GetAnswer();

    ex = new SampleComponent.Example();
}

```